

Early-Layer Locality Is a Transient Requirement, Not an Architectural One

Nicolás Veraz
Independent Researcher
github.com/nicoveraz/neurogen

Abstract

That early transformer layers should attend locally while late layers attend globally is well established: models learn this profile when attention span is made learnable [Sukhbaatar et al., 2019], ablations confirm lower layers do not need long range [Rae and Razavi, 2020], schedules that ramp window size shallow-to-deep improve quality [Xu et al., 2025], and production models ship local-global hybrids [Jiang et al., 2023, Gemma Team, 2025]. **We do not claim this result.** We ask instead *when* the constraint is needed, and find that it is needed only briefly, at the very start. Training a 3.4M-parameter transformer with a quartic depth-wise window schedule for the first 10k of 20k steps and then switching to ordinary full attention retains the entire benefit: +1.46% val_bpb over a paired full-attention baseline across five matched seeds, positive on all five (exact paired permutation $p = 1/32 = 0.031$, paired- t $p = 0.0064$, $d_z = 2.34$), against +1.51% for keeping the windows for the whole run. Whether removal is *better* than keeping them is not resolved and is not resolvable at this measurement precision: only 2 of 5 seeds favour it and the mean difference, -0.05% , is an order of magnitude below the noise. But removal costs nothing, which is the claim with a practical consequence: the locality prior belongs in the training recipe, not in the deployed architecture. How the constraint is released does not matter for quality—widening the windows over 500 steps instead of dropping them at once gives +1.48%, also positive on all five seeds—but it does matter for stability: the abrupt switch triggers a transient shock that lost one run in eight, and the gradual release removes that shock at no measurable cost. *How long* it is needed is the more striking result. Sweeping the release point across seven values from 75% of training down to 1.25%, every one beats its paired baseline on all five seeds, and the effect does not decline as the constraint is shortened—it grows. The best configuration applies the windows for 250 steps of 20,000 and reaches full attention by step 750, yielding +1.91%, the largest gain we measure. We did not find a lower bound. A constraint that is fully released before the model has seen 4% of its data, and whose benefit persists to convergence, is a transient requirement rather than a curriculum, and we have retitled the work accordingly. Two measurements show what the constraint leaves behind. Attention entropy at layer 0 remains 52% below baseline after 100k steps, five times longer than the window phase, so the specialization is a lasting structural feature rather than a transient effect of the mask. Gradient covariance effective rank drops from 48.6 at full attention to 17.2 at window 8, with 96.7% of gradient variance in the top component versus 6.1%—the constraint forces early layers into a coherent, low-dimensional update subspace. A per-layer ablation localizes the effect to layers 0–1 and shows late-layer locality actively hurts, reproducing the known result in our setting. Four further experiments eliminate gradient-noise removal, softmax coupling, and landscape smoothness as mechanisms; implicit regularization remains inconclusive at $n = 2$. At 125M parameters on FineWeb-Edu the windowed schedule gives +2.6% at 20k steps across 5 paired seeds, with one seed negative (paired- t $p = 0.045$, exact paired permutation $p = 0.063$). We also show that the endpoint measurement itself carries a standard deviation of ≈ 0.007 bpb against a 0.0136 effect—which the paired design cancels, but only down to a residual of 0.0023 for the removal arm, and that residual is what puts the removal-versus-retention comparison out of reach. Three further experiments remain pre-registered: a search for the floor below 250 steps, a fixed evaluation set that would make the retention comparison answerable, and a converged 125M three-arm comparison.

1 Introduction

Restricting early transformer layers to a local attention window and letting later layers attend globally is, by now, ordinary practice. It shows up as a learned property when span is made trainable [Sukhbaatar et al., 2019], as an ablation result [Rae and Razavi, 2020], as an explicit depth-wise schedule [Xu et al., 2025], and as a shipped architecture [Jiang et al., 2023, Gemma Team, 2025]. In every one of these treatments the locality is a property *of the model*: it is present at initialization, present throughout training, and present at inference.

This paper asks a different question. If the local-to-global profile is useful, is it useful because the trained network needs it, or because the *training process* needs it? These have different consequences. If the network needs it, the constraint is architectural and must be shipped. If only training needs it, the constraint is transient—something to apply early and then discard, leaving a standard architecture at inference.

We test this directly by removing the windows partway through training. A 3.4M-parameter transformer trained with a quartic depth-wise window schedule for the first half of its step budget, then switched to ordinary full attention for the second half, ends up at least as good as one that kept the windows for the whole run, and better than a full-attention baseline on both seeds that completed. The benefit does not depend on the constraint remaining in place.

Two measurements indicate why. Attention entropy in the early layers stays far below baseline at $5\times$ longer training than the window phase, so the structure the constraint induces is not maintained by the mask—it persists once the mask is gone. And under a restricted window the gradient covariance of the early-layer projections collapses to a low effective rank, so the constraint acts by making early updates coherent rather than by removing noise. A per-layer ablation shows the effect lives in layers 0–1 and that late-layer locality is harmful, which reproduces Rae and Razavi [2020] in our setting and localizes it.

Motivation (not evidence). This work began from an analogy to cortical critical periods, in which constrained connectivity during early development shapes functional architecture that persists after the constraints relax. The analogy is what suggested testing removal rather than presence. It is not tested by any experiment here, and no result below depends on it; we state it once and do not appeal to it again.

Contributions.

- **The locality constraint is removable.** Windows applied for the first 10k of 20k steps and then dropped retain the full benefit over a paired baseline: +1.46%, positive on all five matched seeds, exact paired permutation $p = 0.031$. This reframes a static architectural prior as a transient training-time requirement (§4).
- **Two measurements of what persists.** Early-layer attention entropy stays 52% below baseline at 100k steps; gradient covariance effective rank drops $48.6 \rightarrow 17.2$ under a window-8 constraint (§6).
- **A localization of the known result.** Constraining layers 0–1 alone matches or exceeds the full schedule; constraining only the last layer is worse than baseline (§5).
- **An honest scale probe and a pre-registration.** +2.6% at 125M across 5 paired seeds with one negative, and a stated protocol for the experiments that would make the claim publishable (§7, §8).

2 Related Work

Layer-wise attention range is an established prior. Sukhbaatar et al. [2019] make each

attention head’s span a learned parameter and find that trained models settle on short spans in lower layers and long spans in upper layers—the profile our schedule imposes is one gradient descent discovers on its own. [Rae and Razavi \[2020\]](#) reach the same conclusion by ablation, showing lower layers can be restricted to a short range with no loss. [Xu et al. \[2025\]](#) impose it explicitly: MSWA varies window size across both heads and layers and progressively increases window allocation from shallow to deep layers, reporting gains in both quality and efficiency. Our per-layer ablation (§5) reproduces this and adds nothing to it. We cite it as the setting our result operates in, not as our finding.

Local–global hybrids in production. [Jiang et al. \[2023\]](#) use sliding-window attention in Mistral 7B to handle arbitrary-length sequences at reduced inference cost. [Gemma Team \[2025\]](#) increase the ratio of local to global attention layers and deliberately keep the local span short, as a memory-management strategy for long contexts. Both ship the constraint permanently—at training time and at inference. Our result says the training-time half is what matters, and that the inference-time half can be dropped in settings where the goal is model quality rather than long-context cost.

Curriculum precedents on adjacent axes. The closest prior work is Shortformer [[Press et al., 2021](#)], which trains first on short subsequences and then on longer ones, improving perplexity and cutting training time by $1.65\times$. That is a curriculum over *how much context exists*, staged across a substantial share of training; ours constrains *how much of the available context each layer may use*, at fixed sequence length, and—as §4.1 shows—for only about one percent of it. SWAT [[Fu et al., 2025](#)] trains *with* sliding-window attention specifically to close the gap between windowed inference and full-attention training, retaining windows at inference—the opposite of the deployment recommendation here. [Cabannes et al. \[2025\]](#) vary window size stochastically during training and find it outperforms a fixed window, but again keep the hybrid at inference and target long-context memorization rather than a discardable warmup. To our knowledge no prior work removes a depth-wise locality constraint partway through training and reports that the benefit survives.

Positioning. We do not claim the architecture; we claim the mechanism. That early layers should attend locally and late layers globally is established. Our contribution is that this locality is a property of the training trajectory rather than of the trained model: the constraint can be removed partway through training with no loss, so it belongs in the training recipe, not in the deployed architecture.

Other background. Sliding window attention [[Beltagy et al., 2020](#)] and FlashAttention-2 [[Dao, 2023](#)] supply the efficient implementation our 125M runs use. Structured-initialization work—mimetic initialization [[Trockman and Kolter, 2023](#)], T-Fixup [[Huang et al., 2020](#)], μ P [[Yang et al., 2022](#)—pursues a related goal by constructing weights analytically rather than constraining computation. Induction heads [[Olsson et al., 2022](#)] motivated the pre-wiring variants reported in Appendix A.

3 Method

3.1 Models and data

3.4M model: GPT-style transformer, depth 4, channels 256, 4 heads, 256-token context. Training data: TinyStories [[Eldan and Li, 2023](#)]. Hardware: Apple M1 Pro (MPS). Metric: validation bits-per-byte (val_bpb). Throughput is identical across architectures at 4.8 steps/sec.

125M model: GPT-2 Small architecture (12 layers, 768 dim, 12 heads, 1024 context). Training data: FineWeb-Edu [Penedo et al., 2024]. Hardware: NVIDIA H100 80GB. RoPE, RMSNorm, bfloat16 mixed precision, and FlashAttention-2 with native sliding-window support. Windowed runs are slightly *faster* than baseline (2.83–2.84 vs 2.73–2.78 steps/sec) because the sliding window computes fewer attention scores.

3.2 Depth-wise window schedule

Each layer l (of L) has a maximum attention window

$$w(l) = \min\left(T, b + \left(\frac{l+1}{L}\right)^\gamma (T - b)\right), \quad (1)$$

where T is the sequence length, b is a base floor (8 for the 3.4M model, 16 for 125M), and γ is the growth exponent. Position i attends to $[\max(0, i - w(l) + 1), i]$. For $\gamma = 4$ at depth 4 this gives windows [8, 23, 86, 256]. We searched exponents 0.5–12.0 plus sigmoid, logarithmic, exponential, and Fibonacci schedules; $\gamma \in [3, 4]$ performed best at depth 4.

For the per-layer ablation we also support explicit window lists (e.g. `list:8,23,256,256`), which bypass Eq. 1 and take the listed widths verbatim. All callers at both scales share one schedule implementation, so the 3.4M and 125M operators differ only in the base floor b and the mask dtype.

3.3 Mid-training removal protocol

The removal experiment trains three arms for 20,000 steps each at a fixed seed:

- **A:** full attention throughout.
- **B:** quartic windows throughout.
- **F:** quartic windows for steps 0–9,999, then full attention for steps 10,000–20,000.

The learning-rate schedule is a single cosine over the full 20k steps with a 200-step warmup, identical across arms and *continuous through the switch*—no term in the schedule is keyed to the switch point. Only the attention mask changes at step 10,000. All three arms share the seed, hence the same initialization and the same data order.

3.4 Paired-seed statistics

Every variant shares an initialization with its baseline at the same seed, so the runs are paired and per-seed differences are the unit of analysis. We report the exact one-sided sign-flip permutation test over the paired differences (which floors at $1/2^n$: $1/32 = 0.031$ at $n = 5$, $1/4 = 0.25$ at $n = 2$), the two-sided paired t -test, and the paired effect size d_z . We never use an unpaired test on these data.

This matters because the effect is smaller than the hardware noise floor. Run-to-run variance on MPS for otherwise-identical configurations is ≈ 0.055 bpb, while the paired residual standard deviation for the quartic variant is ≈ 0.004 bpb—roughly $14\times$ smaller. An unpaired comparison at these sample sizes would be uninformative; the paired one is not. Throughout, we report the number of positive paired differences alongside every mean, and any diverged run is reported rather than dropped from the denominator.

4 Result 1: The locality constraint can be removed mid-training

Table 1 gives the per-seed outcome of the protocol in §3.3.

Table 1: Removing windows at the halfway point (20k steps, 3.4M, five matched seeds; init and data order shared within each seed). Arms A and B are the validated runs of Table 12. Arm F removes the windows at once at step 10,000; arm R instead widens them to full linearly over 500 steps. The Δ columns are relative val_bpb improvements over that seed’s own arm A.

Seed	A: full	B: quartic	F: switch	R: ramp	B vs A	F vs A	R vs A
42	0.9041	0.8927	0.8952	0.8950	+1.26%	+0.98%	+1.01%
137	0.8913	0.8788	0.8746	0.8743	+1.40%	+1.87%	+1.91%
256	0.8941	0.8830	0.8865	0.8867	+1.24%	+0.85%	+0.83%
789	0.9098	0.8896	0.8887	0.8882	+2.23%	+2.32%	+2.37%
1337	0.9016	0.8889	0.8901	0.8899	+1.41%	+1.28%	+1.30%
Mean	0.9002	0.8866	0.8870	0.8868	+1.51%	+1.46%	+1.48%

What the data support. Arm F beats arm A on *all five* seeds. The exact one-sided sign-flip permutation test therefore reaches its floor, $p = 1/32 = 0.031$, with paired t -test $p = 0.0064$ and $d_z = 2.34$. The benefit of the locality constraint survives its removal: the windows are not doing ongoing work in the second half of training. This is the claim the paper rests on, and the criterion was fixed before the runs (§8). Arm R, which widens the windows gradually instead, is indistinguishable from arm F and also beats arm A on all five seeds (+1.48%, $p = 0.031$, paired t -test $p = 0.0070$, $d_z = 2.28$). How the constraint is released does not matter for the result; that it is released is what the paper claims.

What the data do not support. Whether F is *better* than B is not resolved, and at this measurement precision it is not resolvable. Only 2 of 5 paired differences favour F; the permutation test returns $p = 0.625$, the paired t -test $p = 0.771$, and $d_z = -0.14$. The mean difference, -0.05% , is an order of magnitude below the residual noise on this comparison (0.0023 bpb, §8). We therefore report “removal costs nothing,” not “removal helps.” An earlier draft of this work reported means of +1.50% for F against +1.33% for B, taken over the two seeds that completed in a three-seed run, and read them as evidence that windows eventually become a ceiling. Those two seeds disagreed in sign; the reading did not survive replication, and the five-seed mean is if anything marginally negative.

Statistical power. The pre-registered criteria were: claim F-vs-A only on 5/5 positive differences, and claim F-vs-B only on $\geq 4/5$ positive *and* paired t -test $p < 0.05$. The first is met; the second fails, as anticipated—given the residual noise of 0.0023 bpb and a true difference no larger than ~ 0.0015 , five seeds gave roughly a 22% chance of meeting it. Resolving F-vs-B needs a lower-variance endpoint measurement, not more seeds; see item (4) of §8.

The switch is a shock, and the divergence is stochastic. An earlier three-seed run of this experiment lost one arm-F run to NaN after the switch, at seed 256. It would be natural to read that as a property of that initialization. It is not: in the five-seed replication of Table 1 the *same* seed completed normally at 0.8865, and no seed diverged. Across both runs the observed rate is one divergence in eight completed arm-F runs, attached to no particular seed.

What every run does take is a large transient shock. Per-step instrumentation around the switch (Table 2) shows a pattern almost invariant across seeds: loss jumps $\approx 3.4\times$, pre-clip

gradient norm jumps 16–20 $\times$, and recovery to pre-switch values completes within about 400 steps.

Table 2: The switch shock, measured per step around step 10,000, across the five seeds of Table 1. Ranges are across seeds. Gradient clipping is at 1.0, so the gradient itself is bounded—the Adam update is not.

	pre-switch	at switch	peak, first 100 steps	by step 10,400
Loss	0.68	2.21–2.37	—	0.67
Gradient norm (pre-clip)	0.136	2.13–2.72	—	0.136
Max Adam update	3.5×10^{-3}	3.6×10^{-3}	$7.5\text{--}7.6 \times 10^{-3}$	3.3×10^{-3}

The peak Adam update is 2.13–2.16 $\times$ its pre-switch value on every one of the five seeds—a spread of 1.4% across seeds, which is what makes it a usable diagnostic where the divergence itself, at one occurrence in eight runs, is not.

Which part of the switch causes which part of the shock. We ran four treatments from a single shared pre-switch state at seed 256, so they differ only in the intervention (Table 3).

Table 3: Switch-shock treatments, all resumed from one identical pre-switch state (seed 256, 1000 post-switch steps). “Recovery” is the number of steps until loss returns below its pre-switch value of 0.68. All treatments reach the same loss by step 10,500 (0.661–0.663).

Treatment	Peak loss	Peak grad norm	Peak Adam update	Recovery
Unmodified switch	2.2112	2.193	7.54×10^{-3}	28
Masked-softmax control	2.2112	2.193	7.54×10^{-3}	28
Optimizer reset	2.2112	2.193	1.56×10^{-3}	68
200-step LR re-warmup	2.2424	2.193	1.90×10^{-3}	72
500-step window ramp	0.7490	0.181	6.71×10^{-3}	1
(pre-switch reference)	0.68	0.136	3.53×10^{-3}	—

Three conclusions follow. First, cause (i) is eliminated: the masked-softmax control agrees with the unmodified switch to 10^{-6} , the rounding precision of the log, at every one of 501 traced steps. Changing the attention implementation contributes nothing. (We note this may be specific to our hardware: PyTorch’s MPS `scaled_dot_product_attention` appears to dispatch to the same arithmetic as the explicit path, so the elimination should not be assumed to transfer to a genuinely fused CUDA kernel.) Second, cause (ii) is confirmed as the *amplifier*: resetting the optimizer leaves loss and gradient spikes untouched but cuts the peak update 4.8 $\times$, to below its pre-switch value—a freshly reset Adam has $\hat{m}/\sqrt{\hat{v}} \approx \pm 1$ on its first step, bounding the update near the learning rate itself. Third, cause (iii) is confirmed as the *source*: ramping the windows to full width over 500 steps, which widens the softmax denominator gradually instead of at once, removes the shock rather than absorbing it—peak loss falls from 2.21 to 0.75 and peak gradient norm from 2.19 to 0.18, below the clipping threshold, so no clipping is triggered at all.

The two absorbing interventions trade update magnitude for time: both cut the peak update roughly fourfold but leave the model in the shocked state $\sim 2.5\times$ longer (28 steps to ~ 70). The ramp pays neither cost and arrives at the same loss.

The ramp is free. Because the shock treatments above are single-seed probes stopped 1000 steps after the switch, they establish what causes the shock but say nothing about converged

quality. We therefore ran the ramp as a full arm—arm R in Table 1—at all five seeds, resuming from the same pre-switch checkpoints arm F used. This is the most tightly controlled comparison in this work: both arms restore an identical model, optimizer and random-number state, so they consume identical training batches and identical evaluation draws, and the endpoint noise that dominates other comparisons cancels. (That the restored stream really does keep two arms aligned is shown by the masked-softmax control above, which tracked the unmodified switch to 10^{-6} over 501 steps despite a different attention configuration.)

Arm R beats the baseline on all five seeds ($p = 0.031$), and against arm F the per-seed differences are $+0.0002$, $+0.0003$, -0.0002 , $+0.0005$, $+0.0002$ bpb: four of five favour the ramp, but the mean difference is $+0.0002$ bpb (0.02%), permutation $p = 0.125$, paired t -test $p = 0.154$. Our pre-registered criterion for claiming the ramp is *better* required $\geq 4/5$ and $p < 0.05$; it is not met, and we do not claim it. What the data do support is that ramping costs nothing. Combined with Table 3, the practical reading is that the gradual release is a free safety improvement: identical converged quality, no loss or gradient spike, and no exposure to the failure mode that cost one run in eight. We recommend it on that basis, not on quality grounds.

The remaining untested claim is the one that motivated the diagnosis: whether ramping actually lowers the divergence rate. One occurrence in eight runs cannot be resolved by five more, and we do not assert it.

The last row is the informative one. Gradient-norm clipping at 1.0 bounds the gradient, and the spike to 2.1–2.7 is duly clipped—but the largest Adam update still climbs $2.1\times$ over the following ~ 100 steps. That is the signature of a stale second moment: v was accumulated under window-8 gradients, so $m/\sqrt{v}$ is large for parameters whose historical v is small, and clipping the gradient does not bound it. Of the three candidate causes we posited—(i) the attention implementation changes at the switch, from an explicit masked softmax to a fused causal kernel; (ii) a stale Adam second moment; (iii) the layer-0 softmax denominator jumping from 8 terms to up to 256 in one step—the trace is direct evidence for (ii) as the amplifier, while (i) and (iii) remain candidates for the initial spike. The learning-rate schedule is ruled out by inspection: a single cosine over the full horizon, with no discontinuity at the switch. §8 specifies the controls that separate the remaining causes. We do not present the recipe as free of this: the naive hard switch is a shock that usually recovers and occasionally does not.

4.1 When to release, and for how long

Table 1 releases the constraint at the halfway mark because that is where we first tried it, not because anything recommended it. To find out whether the release point matters, we swept it over 2k / 5k / 10k / 15k of the 20k-step budget—10% to 75% of training spent windowed—at five seeds each, releasing with the 500-step ramp throughout (Table 4).

Table 4: Release-point sweep (20k steps, 3.4M, five matched seeds, 500-step ramp at each release point). $R@x$ releases the windows at step x . Arm A never applies them; arm B never releases them. Criteria were fixed before the runs (§8).

Seed	A: full	$R@2k$	$R@5k$	$R@10k$	$R@15k$	B: quartic	Best
42	0.9041	0.8932	0.8924	0.8950	0.8956	0.8927	5k
137	0.8913	0.8726	0.8741	0.8743	0.8754	0.8788	2k
256	0.8941	0.8833	0.8847	0.8867	0.8875	0.8830	2k
789	0.9098	0.8855	0.8875	0.8882	0.8890	0.8896	2k
1337	0.9016	0.8863	0.8892	0.8899	0.8915	0.8889	2k
Mean	0.9002	0.8842	0.8856	0.8868	0.8878	0.8866	
vs A	—	+1.78%	+1.62%	+1.48%	+1.38%	+1.51%	

The constraint works at every release point tested. All four columns beat their paired baseline on all five seeds, so each reaches the exact permutation floor $p = 1/32 = 0.031$ (d_z between 2.12 and 2.84). Ten percent of training spent windowed is already enough to obtain the full effect. No run diverged.

Earlier release is better, and the curve is not flat. We pre-registered a null that all release points would fall within the 0.0023 bpb residual of each other, in which case the recipe would need no tuning. That null is falsified: the spread of means is 0.0036 bpb, about a quarter of the baseline-to-quartic effect. The ordering is monotone—earlier release is better—and identical on every seed for the three later points: $R@2k$ beats $R@10k$ and $R@15k$ on 5/5 seeds (paired t -test $p = 0.0026$ and $p = 0.0019$), and $R@10k$ beats $R@15k$ on 5/5 ($p = 0.0049$).

We do not claim an optimum. Our criterion for naming a best release point required the winner to beat *every* other point on $\geq 4/5$ seeds *and* to clear a paired t -test at $p < 0.05$ against the runner-up. $R@2k$ has the best mean, but against $R@5k$ it is 4/5 seeds with $p = 0.0836$, so the criterion is not met and 2k and 5k are not separated by these data. The honest statement is that release should be *early*—somewhere in the first 10–25% of training—and that we cannot resolve where in that range.

Releasing still does not beat retaining. Even the best release point is not distinguishable from keeping the windows throughout: $R@2k$ versus arm B is 3/5 seeds, $p = 0.125$, mean +0.0024 bpb. This is the same conclusion Table 1 reached at the halfway point, and it is the reason the recipe is justified by deployment convenience—a standard architecture at inference—rather than by a quality gain.

How short can it be? Since 2k steps sufficed and shorter was better, we swept further down—1000, 500 and 250 steps, or 5%, 2.5% and 1.25% of training—at the same five seeds. The learning-rate cosine is at its peak at all of these points as well as at 2k, so unlike the 10k-versus-15k comparison these are compared at identical learning rate. The result is that *the benefit never collapses*. Every one of the seven release points beats its paired baseline on all five seeds ($p = 0.031$ throughout, d_z between 2.12 and 2.92), and the shortest is the best: releasing after 250 steps gives +1.91%, the largest effect we measure anywhere in this work. With the 500-step ramp, that model is at full attention from step 750 of 20,000 and trains the remaining 19,250 steps unconstrained.

Table 5: The full release curve (20k steps, 3.4M, five matched seeds). $R@x$ releases the windows at step x ; every point beats its paired baseline on 5/5 seeds. No lower bound was found.

Release at	250	500	1000	2000	5000	10000	15000
% of training	1.25	2.5	5	10	25	50	75
Mean bpb	0.8830	0.8833	0.8846	0.8842	0.8856	0.8868	0.8878
vs baseline	+1.91%	+1.87%	+1.73%	+1.78%	+1.62%	+1.48%	+1.38%
d_z	2.92	2.69	2.57	2.81	2.84	2.28	2.12

This is why the paper is titled as it is. We pre-registered, before running the sweep, that if a release point as early as 250 steps still delivered the full effect then “curriculum” would be the wrong word and the framing would have to change rather than be defended. It did, and it has. A curriculum implies a staged learning process occupying a meaningful share of training; what we observe is a constraint applied to roughly one percent of the optimizer steps, fully released before the model has seen 4% of its data, whose benefit is still present at convergence.

We describe it as a *transient requirement*: needed briefly and early, then not at all. We stop short of calling it an initialization effect, because 250 steps at batch 32 is still two million tokens of training, and because we have not yet found the floor—the curve is flat-to-improving all the way to the smallest point we tested. That search is pre-registered in §8.

One result we do not claim. At $x = 250$ the release arm beats *never releasing* on 4 of 5 seeds with paired- t $p = 0.041$, which would clear the bar we set for that comparison in §4. We reached it, however, by testing seven release points and selecting the best; with seven comparisons the appropriate threshold is nearer $p < 0.007$. We therefore continue to report that releasing does not beat retaining, on multiple-comparisons grounds rather than on the number itself.

Interpretation. That one percent of training suffices, and that longer application is mildly worse, fits the mechanism in §6: the constraint does its work while the hierarchy is being laid down, and after that it is a mild restriction on the upper layers. It is also consistent with the 100k trace in Appendix C, where the baseline-to-quartic gap peaks early and narrows thereafter.

5 Result 2: The active ingredient is early-layer locality

This section reproduces a known result in our setting and localizes it. Sukhbaatar et al. [2019], Rae and Razavi [2020], and Xu et al. [2025] all establish that lower layers want a restricted range while upper layers need global context. We confirm it here in order to identify which part of the schedule the constraint in §4 is actually delivering.

We replaced the quartic schedule with explicit per-layer window lists, training each at seed 42 for 20k steps. Because all configurations share the same initialization *and* the same data order, the only difference is the window pattern (Table 6).

Table 6: Per-layer window ablation (seed 42, 20k steps; matched init and data order). Windows are $[L_0, L_1, L_2, L_3]$; “% of gain” is the fraction of the baseline→quartic gap (0.0114 bpb) recovered.

Config	Windows	Final bpb	vs Baseline	% of gain
Baseline	[256, 256, 256, 256]	0.9041	—	—
Only L_0	[8, 256, 256, 256]	0.8952	+0.98%	78%
Only L_0, L_1	[8, 23, 256, 256]	0.8880	+ 1.79%	141%
Quartic	[8, 23, 86, 256]	0.8927	+1.26%	100%
No L_0	[256, 23, 86, 256]	0.8979	+0.69%	54%
Only L_3 (reversed)	[256, 256, 256, 8]	0.9127	−0.95%	−75%

Windowing layer 0 alone recovers 78% of the quartic gain; windowing the first two layers and leaving L_2, L_3 at full attention exceeds the full schedule; and the reversed control—windowing only the last layer—is *worse* than baseline. It is specifically early locality that helps, not locality per se. A practical corollary: the gradual quartic ramp is not essential. “Early layers local, rest full” matches or beats it, which is consistent with hybrid designs in which the exact schedule shape matters little [Gemma Team, 2025, Xu et al., 2025].

Caveat. This is a single seed. The large effects—early-local recovers most of the gain, the reversed control hurts—are robust to the noise floor. The modest margin by which Only L_0, L_1 beats Quartic (0.0047 bpb) is not, and should be replicated before being asserted; §8 pre-registers the criterion.

6 What the constraint leaves behind

If removing the windows preserves the benefit, something the windows created must persist without them. Two measurements characterize it.

6.1 Attention entropy persists at $5\times$ longer training

We measured per-layer attention entropy $H = -\sum_j \alpha_{ij} \log \alpha_{ij}$ on validation data (100 batches), averaged over heads, query positions, and batches, for baseline and quartic checkpoints at both 20k and 100k steps (Table 7).

Table 7: Attention entropy per layer at 20k and 100k steps (seed 42, 100 batches).

Layer	20k steps			100k steps		
	Baseline	Quartic	Change	Baseline	Quartic	Change
0	1.994	0.852	−57.3%	1.860	0.898	−51.7%
1	1.506	0.916	−39.2%	1.933	1.248	−35.4%
2	2.466	2.183	−11.5%	2.722	2.489	−8.6%
3	2.120	2.449	+15.5%	2.382	2.605	+9.4%

At 100k steps the quartic model still shows 52% lower entropy at layer 0 and 35% lower at layer 1; the gap narrows only slightly from 20k. The final layer consistently shows *higher* entropy, using its full span to integrate globally over the local features built below. Effective attention spans at 100k confirm the locality is learned rather than merely masked: quartic layer 0 attends to ~ 2 of its 8 permitted tokens, while baseline layer 0 attends to ~ 8 of 256. The structure is a property of the weights, not of the mask—which is what §4 predicts.

6.2 Gradient covariance rank: the constraint makes early updates coherent

On a frozen baseline checkpoint we collected 50 gradient samples for the layer-0 Q/K weights at five window sizes and computed the SVD of the gradient matrix (Table 8).

Table 8: Gradient covariance rank vs window size (layer 0, Q/K weights, 50 passes).

Window	Effective rank	Var. top-1	Var. top-5
8	17.2	96.7%	97.3%
32	45.5	34.0%	45.3%
64	48.4	9.2%	25.2%
128	48.6	5.9%	23.0%
256	48.6	6.1%	22.8%

Effective rank drops from 48.6 at full attention to 17.2 at window 8—a $2.8\times$ reduction—and 96.7% of gradient variance concentrates in the top singular component versus 6.1% at window 256. Under the constraint, each gradient step updates a coherent low-dimensional subspace rather than a diffuse high-dimensional one. This is a plausible route by which an early constraint fixes a hierarchy that then persists, though we have not shown the causal link directly.

A complementary measurement points the same way. On a frozen trained checkpoint, gradient noise norm is approximately constant across window sizes (0.0052–0.0058), while signal norm rises $18\times$ from window 256 (0.0017) to window 8 (0.0323). Windows increase gradient *coherence*; they do not reduce noise.

6.3 Hypotheses eliminated, and one that is not

Table 9 summarizes four mechanism experiments that rule alternatives out, plus one that does not resolve.

Table 9: Mechanism hypotheses tested. Experiment numbering follows the repository.

Hypothesis	Evidence	Verdict
Gradient noise removal (Exp 1)	Noise norm constant 0.0052–0.0058 across windows 8–256; only signal changes	Eliminated
Softmax coupling contamination (Exp 2)	Noise fraction 4.0–6.9% across all 4 layers	Eliminated
Variance reduction (Exp 3)	Larger batch cannot replicate the effect at equal tokens	Eliminated ($n = 1$)
Landscape smoothness (Exp 6)	Quartic-trained models have <i>lower</i> gradient stability on both seeds (0.0335 vs 0.0358 at seed 42; 0.1101 vs 0.1121 at seed 137)	Eliminated
Ongoing structural constraint (Exp 7)	Removing windows preserves the benefit (§4)	Eliminated
Implicit regularization (Exp 4)	Seed 42: quartic gap <i>larger</i> (+2.48% vs +0.32%). Seed 137: quartic gap <i>smaller</i> (−2.70% vs +0.80%). Seeds contradict	Inconclusive at $n = 2$

On the variance-reduction control (Exp 3). If windows worked by reducing gradient variance, larger batches should replicate them. At 2000 optimizer steps, seed 42 only: full attention at batch 32 reaches 1.242 bpb (16.4M tokens), quartic windows at batch 32 reach 1.224 (+1.45%, same tokens), and full attention at batch 128 reaches 1.086 but on $4\times$ the data (65.5M tokens). Compared at equal tokens (16M), the batch-128 run is at 1.357—worse than either batch-32 run. Larger batches look better only because they saw more data; at equal token budget the windowed run wins and the larger-batch control loses. This is a single seed and we report it as such. A batch-256 arm was launched but did not complete its step budget and is excluded.

On implicit regularization (Exp 4). We measured train and validation bpb on frozen checkpoints to compute the generalization gap. The two available seeds disagree in direction. At $n = 2$ with contradictory signs this experiment neither rules implicit regularization in nor out, and we do not count it among the eliminations. An earlier draft reported only the seed-42 direction and claimed a clean elimination.

7 Scale: 125M parameters

Table 10 gives the per-seed gap at a matched 20k steps across all five seeds. Each seed’s baseline and quartic arms share an LR schedule, so the within-seed gap is valid even though seeds 42/137 run a 50k schedule read at step 20k while 256/789/1337 are dedicated 20k runs.

Table 10: 125M per-seed gap at a matched 20k steps (all 5 seeds, paired).

Seed	Baseline bpb	Quartic bpb	Gap
42	4.101	3.894	+5.04%
137	4.090	3.989	+2.47%
256	3.696	3.704	− 0.21%
789	3.578	3.503	+2.09%
1337	3.620	3.492	+3.53%
Mean	—	—	+ 2.6% (sd 1.94)

Four of five seeds are positive; seed 256 is negative. The exact one-sided sign-flip permutation test over the five paired differences gives $p = 2/32 = 0.063$, the paired t -test $p = 0.045$, and $d_z = 1.29$. This is a weaker result than the 3.4M one (where all 5 seeds were positive and the permutation test hit its floor of 0.031), and we report it as a suggestive scale probe rather than a demonstrated scaling law.

Convergence. Neither 50k-step arm is converged. Over the final 10k steps the baselines are still descending at 0.0098 (seed 42) and 0.0084 (seed 137) bpb per 1k steps, and the quartic arms are descending *faster* still (0.0208 and 0.0166). A larger gap at 50k is therefore what an unconverged pair of curves produces regardless of whether the asymptotic gap is larger, so we do not read the 50k numbers as a scaling result. They are reported in Appendix B with that label. Determining whether the gap is real requires a single fully-annealed horizon; the protocol is in §8.

Throughput. Windowed 125M runs are slightly faster than baseline (2.83–2.84 vs 2.73–2.78 steps/sec), because FlashAttention’s sliding window computes fewer attention scores. Under this recipe the advantage applies only to the brief windowed phase, and inference runs at standard full-attention cost.

8 Limitations and pre-registered experiments

8.1 Limitations

- **F vs B is unresolved and unresolvable here** (2/5 seeds, $p = 0.625$). The residual noise on that comparison, 0.0023 bpb, exceeds any plausible true difference; more seeds will not fix it, a lower-variance endpoint would. It is not claimed (§4).
- **Arms A and B are reused, not re-run.** Table 1 pairs a freshly trained arm F against the cached arms of Table 12. This is what makes the comparison affordable, but it means A and B carry the eval draws of their original runs; the residual noise figure above already accounts for it.
- **The hard switch is a shock that occasionally kills a run** (one divergence in six completed arm-F runs). It is stochastic rather than tied to a particular seed, and the stale-Adam-second-moment mechanism is evidenced but not yet isolated from the other two candidates.
- **One switch point only.** We removed windows at the 50% mark. Whether 10%, 25%, or 75% is better—or whether it matters at all—is untested, and this is what would turn the finding into an actionable recipe.

- **The per-layer ablation is a single seed**; the Only L_0, L_1 vs Quartic margin needs replication.
- **The variance-reduction control (Exp 3) is a single seed**, and its batch-256 arm did not complete.
- **Implicit regularization is unresolved** (seed-contradictory at $n = 2$).
- **125M evidence is a 20k-step probe**, not a converged comparison, and one of five seeds is negative. The removal experiment has not been run at 125M at all.
- **The endpoint measurement is noisier than the effect.** A single evaluation over 12 randomly drawn batches has a standard deviation of ≈ 0.007 bpb on a frozen checkpoint, against an effect of 0.0136. Paired comparison cancels most of it (residual 0.0010 within a script, 0.0023 across scripts), which is why every claim here is paired and why an unpaired test on these data would be uninformative. A fixed evaluation set would remove the term entirely; see item (4) of the pre-registration.
- **Mechanism experiments were performed only at 3.4M.**
- **No downstream task evaluation** (CORE, MMLU, etc.); the 3.4M model uses a narrow domain (TinyStories).
- **The optimal exponent at depth 12 is undetermined**—only $\gamma = 4$ was run at 125M.
- **A second-order probe of landscape flatness** (Hessian trace and top eigenvalue) on one converged matched pair was inconclusive at $n = 1$ and is not reported as a result. The attention-entropy/sharpness relationship is itself an established area; a credible test requires multiple converged pairs at scale.

8.2 Pre-registered experiments

We state criteria in advance so that outcomes cannot be re-framed after the fact.

(1) Removal at five seeds — completed. Criteria fixed before the runs: claim “removal preserves the benefit” iff all 5 paired differences ($A - F$) are positive, giving the exact permutation floor $p = 0.031$; claim “removal is better than keeping” only if at least 4 of 5 paired differences ($B - F$) are positive *and* the paired t -test gives $p < 0.05$; diverged seeds count in the denominator. *Outcome*: the first criterion is met (5/5, $p = 0.031$, paired- t $p = 0.0064$, $d_z = 2.34$); the second fails (2/5, $p = 0.625$). Reported in Table 1. No seed diverged.

(2) Shock diagnosis — completed. Because the divergence occurs roughly once in eight runs and is tied to no seed, an experiment keyed to reproducing the NaN would be badly powered; the criterion instead targeted the shock, which reproduces on every seed. Fixed in advance: run five treatments from one shared pre-switch state and report which reduces the peak Adam update toward its pre-switch value of 3.5×10^{-3} , with optimizer reset predicted to do so and the stale-second-moment account to be withdrawn if it did not. *Outcome*: optimizer reset cut the peak update $4.8\times$, so the account stands; the kernel-change cause was eliminated; and the window ramp, included only as a candidate mitigation, turned out to remove the shock at its source rather than absorb it. Reported in Table 3. What remains untested is whether any treatment improves converged val_bpb or lowers the divergence rate—see the limitations above.

(2b) Ramp arm to convergence — completed. Criteria fixed in advance: claim “the ramped curriculum preserves the benefit” iff all 5 paired differences ($A - R$) are positive; claim “ramping is better than switching” only if $\geq 4/5$ paired differences ($F - R$) are positive *and* the paired t -test gives $p < 0.05$; and withdraw the ramp recommendation entirely if R is significantly *worse* than F . *Outcome*: the first is met (5/5, $p = 0.031$, $d_z = 2.28$); the second is not (4/5 but $p = 0.154$) and is not claimed; the withdrawal condition did not trigger. Reported in Table 1.

(3) Release-point sweep — completed. Criteria fixed in advance: claim the constraint works at a release point iff all 5 paired differences are positive; claim an optimal release point only if the winner beats every other point on $\geq 4/5$ seeds *and* clears paired- t $p < 0.05$ against the runner-up; and treat a flat curve (all points within the 0.0023 bpb residual) as a result rather than a failure, since it would mean the recipe needs no tuning. *Outcome*: the first is met at all four points (5/5, $p = 0.031$); the flat null is falsified (spread 0.0036 bpb); the optimum criterion is *not* met, since $R@2k$ versus $R@5k$ is 4/5 with $p = 0.0836$. Reported in Table 4. Original text follows for the record. Release the windows at 2k / 5k / 10k / 15k of a 20k-step run, at 5 seeds, using the 500-step ramp rather than a hard switch. The endpoints $x = 0$ and $x = 20k$ are arms A and B. *Criterion*: claim an optimal removal point only if the best interior x beats *both* endpoints on at least 4 of 5 paired seeds. If the curve is flat in x , the finding is that the removal point does not matter over 10–75% of training—which is a stronger recipe, since it requires no tuning. Report the divergence rate per switch point.

(3b) Where is the floor? — running. The release curve is flat-to-improving down to 250 steps, so the minimum duration is unknown. Release at 50 / 100 / 250 steps with the ramp set equal to the release point, so the model reaches full attention at step $2x$ and a fixed ramp cannot swamp the swept variable; $x = 250$ is re-run at ramp 250 to bridge to the existing $x = 250$, ramp 500 result. *Criterion*: claim the constraint works at x iff all 5 paired differences are positive; report the smallest x that passes. *Second pre-registered reframe*: if $x = 50$ works—full attention from step 100 of 20,000, before the 200-step learning-rate warmup has finished—then “transient requirement” is itself too weak and *initialization effect* becomes the accurate description, since the constraint would be shaping only the first few dozen updates. We commit to that reading in advance, as we did for the present retitling.

(4) Fixed evaluation set. Every reported endpoint is a single evaluation over 12 randomly drawn batches (98,304 tokens, 0.51% of the validation set), resampled on each call. Evaluating one *frozen* checkpoint twelve times gives a standard deviation of 0.0083 (baseline, seed 42) and 0.0063 (quartic), with a range up to 0.030—over half the size of the 0.0136 effect. The paired design rescues this: paired arms consume the random number stream near-identically and therefore draw nearly the same evaluation batches, so the noise is common-mode. Detrended evaluation fluctuations correlate at $r = 0.989$ between the two arms produced by the same script, leaving a residual difference standard deviation of 0.0010. Arm F is produced by a different script that consumes the stream slightly differently, giving $r = 0.947$ and a residual of 0.0023. Against that residual, F vs A is 3.9σ and 7.6σ and B vs A is $11\text{--}20\sigma$, but F vs B is 0.8σ and 2.1σ —indistinguishable from noise, which is the quantitative reason the sign split in Table 1 cannot be resolved. *Change*: evaluate on a deterministic set of offsets built once from a private random generator (so it draws nothing from the global stream and leaves training data order untouched), sized at $\approx 1M$ tokens. *Criterion*: repeated evaluation of one frozen checkpoint must return bit-identical values, i.e. standard deviation exactly zero. This changes the recorded endpoint of every run, so all arms must be re-measured together rather than compared against existing numbers. *Consequence for (1)*: at the current residual, $n = 5$ gives roughly a 22% chance of five positive differences and an expected paired- t $p \approx 0.22$ for F

vs B , so experiment (1) is not expected to resolve it; reaching $p < 0.05$ at $n = 5$ requires the residual below ≈ 0.0012 .

(5) Converged 125M, three arms, one horizon. Baseline / windows-throughout / windows-removed-at-25k, at 50k steps (6.55B tokens, ≈ 53 tokens per parameter, comfortably past the compute-optimal ratio of Hoffmann et al. [2022]), under a single fully-annealed cosine with no resume, at 3–5 seeds. Cost is ≈ 5 H100-hours per run, so ≈ 45 H100-hours at 3 seeds and ≈ 75 at 5. *Criterion:* a run counts as converged only if its decline over the final 10k steps is below 0.002 bpb per 1k steps; report the measured value for every run. Note that at $n = 3$ the exact permutation test floors at 0.125 and cannot reach conventional significance, so a 3-seed result is reported descriptively with the per-seed table. *We pre-register the negative outcome:* if the converged gap is smaller than the 20k gap, we will report that the 50k figures in Appendix B were an undertraining artifact.

9 Discussion

The practical recipe. Apply a depth-wise locality constraint for the first 10–25% of training, then widen the windows to full over a few hundred steps and train normally. A tenth of the step budget is enough (§4.1); applying it for longer is if anything mildly worse. The shipped model is a standard transformer with no architectural modification, no extra parameters, and standard inference cost. Removing the constraint abruptly gives the same converged quality (Table 1) but incurs a large transient shock and, in our runs, a one-in-eight chance of losing the run outright; the ramp costs nothing and avoids it. This differs from every local–global hybrid in production [Jiang et al., 2023, Gemma Team, 2025] and from SWAT [Fu et al., 2025], all of which retain windows at inference, and it means the technique composes with whatever attention implementation a deployment already uses.

Transient, not architectural. The result that makes this a transient requirement rather than an architectural one is the persistence in Table 7: the low early-layer entropy the constraint induces survives $5\times$ longer training. The model does not revert when the mask is lifted. Combined with the gradient covariance collapse under a restricted window, the picture is that the constraint makes early-layer updates coherent during the phase when the hierarchy is being laid down, and that the resulting hierarchy is then self-sustaining. We have shown the persistence and the coherence separately; we have not shown that the second causes the first.

What would falsify this. A flat or negative release-point sweep at five seeds, or a converged 125M comparison in which the released arm underperforms the windowed arm, would each undercut the recipe. The criteria are fixed in §8 before the runs. The recipe would also be undercut if the release point turned out to matter sharply, since we have tested only one—the halfway mark.

10 Conclusion

Early-layer locality is a well-established prior for transformers, and we do not claim it. We claim that it is needed only during early training. Removing a quartic depth-wise window schedule at the halfway point of a 20k-step run retains the benefit over a paired full-attention baseline: +1.46% across five matched seeds, positive on every one (exact paired permutation $p = 0.031$, paired- t $p = 0.0064$, $d_z = 2.34$), against +1.51% for keeping the windows for the whole run. Whether removal is *better* than keeping is not claimed: 2 of 5 seeds favour it, and the difference is an order of magnitude below the residual measurement noise. What the constraint leaves behind

is measurable: early-layer attention entropy stays 52% below baseline at $5\times$ longer training, and under a restricted window the early-layer gradient covariance collapses from effective rank 48.6 to 17.2 with 96.7% of variance in a single component. A per-layer ablation puts the effect in layers 0–1 and shows late-layer locality hurts. At 125M the schedule gives +2.6% at 20k across 5 paired seeds with one negative. The consequence is that this prior belongs in the training recipe rather than the deployed architecture: use windows as warmup, ship a standard transformer. How the constraint is released does not affect the result—removing the windows at once and widening them over 500 steps both give $\approx +1.5\%$ on all five seeds—but it does affect the risk. The hard switch produces a transient shock that usually recovers within ~ 30 steps and, once in eight runs, does not recover at all; the ramp removes that shock at no measurable cost in quality. The recipe is therefore to ramp rather than switch, on safety grounds rather than quality grounds.

Code and data: <https://github.com/nicoveraz/neurogen>

A The broader search this result came from

The window schedule was not designed; it was the surviving candidate from an autonomous search over 200+ fixed-budget experiments following the autoresearch paradigm [Karpathy, 2026], in which an agent modifies training code, runs experiments, keeps improvements, and discards failures. Twenty-six approaches were organized into four categories: **(A) weight decoration**—cellular-automata initialization (grid, block-diagonal, reaction-diffusion, spectral), live CA during training, embryogenic activity-dependent CA, pre-wired circuits [Olsson et al., 2022, Anthropic, 2025, Mordvintsev et al., 2020, Najarro and Risi, 2022]; **(B) architectural constraints**—the attention window schedules studied here, CA-generated attention bias; **(C) combinations**; **(D) radical modifications**—CA modulation channels, token vitality, sleep consolidation, cross-layer CA state, developmental dropout.

Table 11 records the negative results. They are reported for completeness and are not evidence for any claim in the main text.

Table 11: Summary of unsuccessful approaches from the search.

Approach	Best result	Experiments	Failure mode
CA initialization	+0.6%	40+	Marginal; simpler patterns > complex
Live CA	−2 to −5%	15+	Per-step overhead kills performance
Embryogenic CA	+1.3%	16	Modest; high overhead
Pre-wired circuits alone	−3 to −5%	12	Gradients destroy pre-wired structure
CA modulation channels	divergence	8	Model collapse
Token vitality	divergence	4	Model collapse
Sleep consolidation	−1%	6	Overhead > benefit

For reference, the validated 3.4M window variants at 20k steps across 5 matched seeds are in Table 12. All five paired differences are positive for every variant, so the exact permutation test reaches its $1/32$ floor in each case; the paired t and d_z separate them.

Table 12: 3.4M validation results (20,000 steps, 5 matched seeds). p_{perm} is the exact one-sided sign-flip permutation p (floor $1/32$); p_t is the two-sided paired t -test; d_z is the paired effect size.

Config	n	Mean bpb	Std	vs Baseline	p_{perm}	p_t	d_z
Baseline	5	0.9002	0.0075	—	—	—	—
Quadratic ($\gamma = 2$)	5	0.8911	0.0048	+1.0%	0.031	0.012	1.94
Quartic ($\gamma = 4$)	5	0.8866	0.0056	+1.5%	0.031	0.001	3.59
Quad + Induction	5	0.8899	0.0041	+1.1%	0.031	0.009	2.09

Note that combining the schedule with pre-wired induction heads reaches only +1.1%—adding the scaffold does not help beyond the constraint itself.

B 125M extension to 50k steps (unconverged; suggestive only)

These numbers do not support a scaling claim and are reported only for completeness. Two of the five 125M seeds were extended to 50k steps. Neither arm satisfies the convergence criterion of §8: over the final 10k steps the baselines decline 0.0098 and 0.0084 bpb per 1k steps and the quartic arms decline 0.0208 and 0.0166—all an order of magnitude above the 0.002 threshold, with the quartic arms descending faster than the baselines. A gap that widens between two curves that are both still falling is uninformative about the asymptotic gap.

Table 13: 125M gap evolution on the two seeds extended to 50k (mean of seeds 42 and 137). **Both arms unconverged at every row.**

Steps	Baseline bpb	Quartic bpb	Gap
5k	4.911	4.820	+1.9%
10k	4.580	4.390	+4.2%
20k	4.095	3.942	+3.8%
30k	3.844	3.576	+7.0%
40k	3.538	3.188	+9.9%
50k	3.447	3.001	+12.9%

Per-seed at 50k the gaps are +17.2% (seed 42) and +8.4% (seed 137), i.e. $n = 2$ with a factor-of-two spread. The headline 125M number remains +2.6% at 20k across 5 seeds (§7).

C 3.4M extended training to 100k steps

To check that the advantage is not merely faster early convergence, both baseline and quartic were trained for 100k steps at seed 42 (Table 14).

Table 14: 3.4M gap evolution over 100k steps (seed 42, $n = 1$).

Steps	Baseline bpb	Quartic bpb	Gap	Note
1k	1.339	1.326	+1.0%	
5k	1.122	1.109	+1.2%	
10k	1.051	1.021	+2.9%	peak gap
20k	0.971	0.957	+1.4%	
50k	0.898	0.893	+0.6%	
100k	0.807	0.799	+1.0%	persists

The gap peaks early (+2.9% at 10k), narrows during mid-training, and does not close by 100k. The best checkpoint seen for each arm is 0.7921 (quartic, 96k) versus 0.7980 (baseline, 96k). This is a single seed, and the run-to-run noise floor on this hardware (≈ 0.055 bpb) exceeds the final gap, so the endpoint value should not be read as a measured effect size; the paired 20k results in Table 12 carry that. What this run supports is the qualitative shape—an early peak consistent with a curriculum effect, and no reversion at long horizons, which is the same conclusion the entropy measurements reach in Table 7.

References

- Anthropic. Circuit Tracing: Revealing Computational Graphs in Language Models. *Transformer Circuits Thread*, 2025.
- I. Beltagy, M. E. Peters, and A. Cohan. Longformer: The Long-Document Transformer. *arXiv:2004.05150*, 2020.
- L. Cabannes, M. Beck, G. Szilvasy, M. Douze, M. Lomeli, J. Copet, P.-E. Mazaré, G. Synnaeve, and H. Jégou. Short Window Attention Enables Long-Term Memorization. *arXiv:2509.24552*, 2025.
- T. Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *arXiv:2307.08691*, 2023.
- R. Eldan and Y. Li. TinyStories: How Small Can Language Models Be and Still Speak Coherent English? *arXiv:2305.07759*, 2023.
- Z. Fu, W. Song, Y. Wang, X. Wu, Y. Zheng, Y. Zhang, D. Xu, X. Wei, T. Xu, and X. Zhao. Sliding Window Attention Training for Efficient Large Language Models. *arXiv:2502.18845*, 2025.
- Gemma Team, Google DeepMind. Gemma 3 Technical Report. *arXiv:2503.19786*, 2025.
- J. Hoffmann, S. Borgeaud, A. Mensch, et al. Training Compute-Optimal Large Language Models. *arXiv:2203.15556*, 2022.
- X. S. Huang, F. Perez, J. Ba, and M. Volkovs. Improving Transformer Optimization Through Better Initialization. *ICML*, 2020.
- A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, et al. Mistral 7B. *arXiv:2310.06825*, 2023.
- A. Karpathy. nanochat / autoresearch. <https://github.com/karpathy/autoresearch>, 2026.
- A. Mordvintsev, E. Randazzo, E. Niklasson, and M. Levin. Growing Neural Cellular Automata. *Distill*, 2020.
- E. Najarro and S. Risi. HyperNCA: Growing Neural Cellular Automata to Grow Neural Networks. *arXiv:2204.11674*, 2022.
- C. Olsson, N. Elhage, N. Nanda, et al. In-context Learning and Induction Heads. *Transformer Circuits Thread*, 2022.
- G. Penedo, H. Malartic, D. Hesslow, et al. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. *arXiv:2406.17557*, 2024.
- O. Press, N. A. Smith, and M. Lewis. Shortformer: Better Language Modeling using Shorter Inputs. In *Proceedings of ACL*, 2021. *arXiv:2012.15832*.

- J. W. Rae and A. Razavi. Do Transformers Need Deep Long-Range Memory? In *Proceedings of ACL*, 2020. arXiv:2007.03356.
- S. Sukhbaatar, E. Grave, P. Bojanowski, and A. Joulin. Adaptive Attention Span in Transformers. In *Proceedings of ACL*, 2019. arXiv:1905.07799.
- A. Trockman and J. Z. Kolter. Mimetic Initialization of Self-Attention Layers. *arXiv:2305.09828*, 2023.
- Y. Xu, S. Nag, D. Li, L. Tian, and E. Barsoum. MSWA: Refining Local Attention with Multi-Scale Window Attention. *arXiv:2501.01039*, 2025.
- G. Yang, E. J. Hu, I. Babuschkin, et al. Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer. *arXiv:2203.03466*, 2022.